核心结论

- `transform` **不会触发布局（reflow / layout）**，因为它**不改变元素在文档流中的占位和几何尺寸**。
- 一般情况下也**不需要重新光栅化整个页面的像素（大规模 repaint）**，而是只在**合成阶段（composite）做图层的位移/旋转/缩放**。

浏览器渲染流程简化版

从 CSS 属性到屏幕，大致是这条流水线：

1. **Style**：计算样式
2. **Layout（重排）**：计算每个盒子的大小、位置
3. **Paint（重绘）**：把每个盒子画成位图（像素）
4. **Composite（合成）**：把各个图层在 GPU 上叠加、变换，输出到屏幕

`transform` 的“特殊待遇”

- 改 `left/top/width/height` 等几何属性：
  - 会改变盒子尺寸或位置 → 影响布局
  - 触发：**Layout → Paint → Composite**
- 改 `transform`：

```
.box {
  transform: translateX(100px) scale(1.2);
}

```

  - 元素在文档流中的“占位矩形”不变
  - 浏览器通常把它放到一个**单独的合成层（compositing layer）**
  - 对这个图层做**矩阵变换**：平移、缩放、旋转都在 **Composite 阶段由 GPU 完成**
  - 流水线只需：**Composite（合成）**，不必重新走 Layout / 大规模 Paint

为什么说它“不会重排重绘”？

严格一点说：

- **肯定不会重排（Layout）**：不改布局信息
- **通常不会触发昂贵的重绘（全量 Paint）**：
  - 元素内容已被光栅化为位图
  - `transform` 只是拿这块位图“搬来搬去、放大缩小”，属于**合成操作**

> 因此前端里常说：**动画尽量用 `transform` 和 `opacity`，因为它们只走合成阶段，性能最好。**